



República De Panamá
Universidad Tecnológica De Panamá
Centro Regional De Azuero



Facultad De Ingeniería En Sistemas Computacionales
Lic. En Ingeniería Sistemas y Computación

Prof. Marisol Quintero

Lenguaje de Programación I

Proyecto Semestral

Laberinto

Miguel Oliver 8-1050-1381

Juan Rodríguez 6-728-1695

Alejandra Falcón 8-1029-738

Nieves Pérez 6-728-1017

Fecha de entrega:

22/07/26

Año lectivo 2026- I Semestre

Introducción

La programación de software interactivo requiere no solo el dominio de la sintaxis de un lenguaje, sino también una profunda comprensión sobre cómo gestionar, almacenar y manipular datos de forma eficiente para simular entornos lógicos. Bajo esta premisa surge el presente proyecto, titulado "Laberinto Interactivo con Niveles", una aplicación de escritorio desarrollada íntegramente en el lenguaje C# bajo el framework de .NET (Windows Forms). El objetivo principal de este trabajo es crear una experiencia lúdica y dinámica donde el usuario deba emplear su razonamiento espacial y su agilidad en la toma de decisiones para navegar desde un punto de origen hasta una meta, superando diversos laberintos de dificultad progresiva.

Para lograr este cometido, el núcleo lógico de la aplicación se cimentó sobre el uso intensivo de estructuras de datos matemáticas, destacando el manejo de arreglos o matrices bidimensionales que actúan como la columna vertebral del mundo virtual. Al mapear gráficamente estas cuadrículas numéricas, el programa es capaz de interpretar paredes, pasillos y elementos interactivos, resolviendo el desafío algorítmico de validar colisiones y movimientos en tiempo real. Adicionalmente, el diseño del sistema integra los principios de la Programación Orientada a Objetos (POO), garantizando una arquitectura de código modular donde clases totalmente independientes asumen responsabilidades específicas: desde el renderizado de gráficos y el procesamiento de eventos de hardware (teclado/ratón), hasta la supervisión de las estadísticas del jugador (cronómetro, número de pasos y coleccionables).

En el presente documento se detalla exhaustivamente la metodología y la lógica detrás del desarrollo, demostrando con evidencias técnicas cómo la aplicación satisface de manera estricta todos los requerimientos solicitados para este proyecto final. Asimismo, se expone la implementación de un robusto conjunto de mecánicas adicionales—tales como algoritmos de búsqueda y persecución para enemigos, efectos audiovisuales y un sistema de niveles con condiciones de victoria variadas—que fueron integradas con el firme propósito de enriquecer la interacción del usuario y exhibir un alto nivel de dominio en la lógica algorítmica y la creación de aplicaciones en C#.

Matriz [filas, columnas] y Manejo de Arreglos

Explicación: El corazón del mapa reside en la clase `Level.cs`. Para representar la estructura del laberinto, se utiliza una matriz bidimensional (`int[,] Grid`). Este enfoque matricial permite que cada celda de la pantalla represente un valor numérico: 0 para caminos y 1 para paredes. El motor gráfico recorre esta matriz mediante ciclos `for` anidados para dibujar el entorno.

Evidencia de Código (Renderizado desde la Matriz en `FormJuego.cs`):

C#

```
private void pbMaze_Paint(object sender, PaintEventArgs e)
{
    // Iteración sobre el arreglo bidimensional
    for (int y = 0; y < _gameManager.CurrentLevel.Rows; y++)
    {
        for (int x = 0; x < _gameManager.CurrentLevel.Columns; x++)
        {
            int cellValue = _gameManager.CurrentLevel.Grid[y, x];
            Rectangle rect = new Rectangle(x * CellSize, y * CellSize, CellSize, CellSize);

            if (cellValue == 1) // Si el valor en la matriz es 1, dibuja pared
                e.Graphics.DrawImage(_wallTexture, rect);
            else if (cellValue == 2) // Si es 2, dibuja un apunte a recolectar
                e.Graphics.DrawImage(Properties.Resources.apunte, rect);
        }
    }
}
```

Implementacion de al menos 3 Niveles (Dificultad Progresiva)

Explicación: El programa incluye los 3 niveles obligatorios, aumentando en tamaño y obstáculos.

- Nivel 1 (Laberinto pequeño): Matriz de 10x10. Se enfoca en enseñar controles mediante un panel gráfico de Tutorial.
- Nivel 2 (Más obstáculos): Matriz de 15x15. Introduce rutas no lineales.
- Nivel 3 (Camino engañosos): Matriz gigante de 20x20 llena de callejones sin salida.

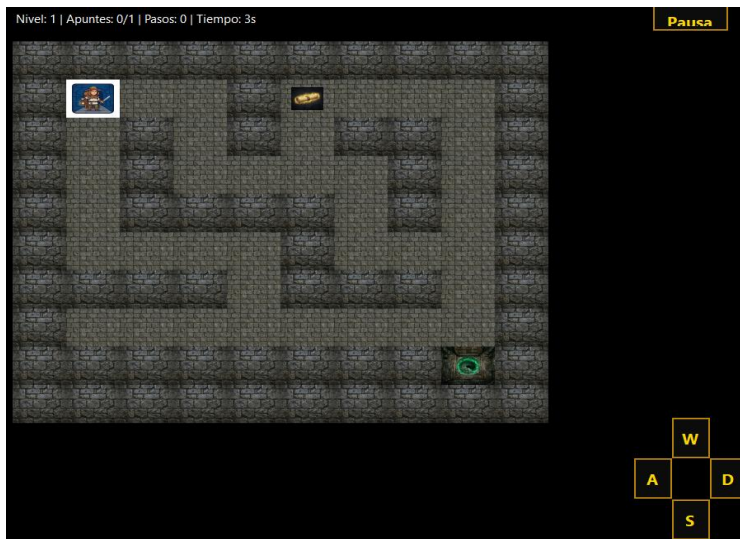


Figura 1: Vista de jugabilidad del Nivel 1 (Fase Tutorial). Se observa el diseño en matriz de 10x10, el avatar del jugador y la interfaz de botones direccionales (W, A, S, D) que actúan como guía inicial.

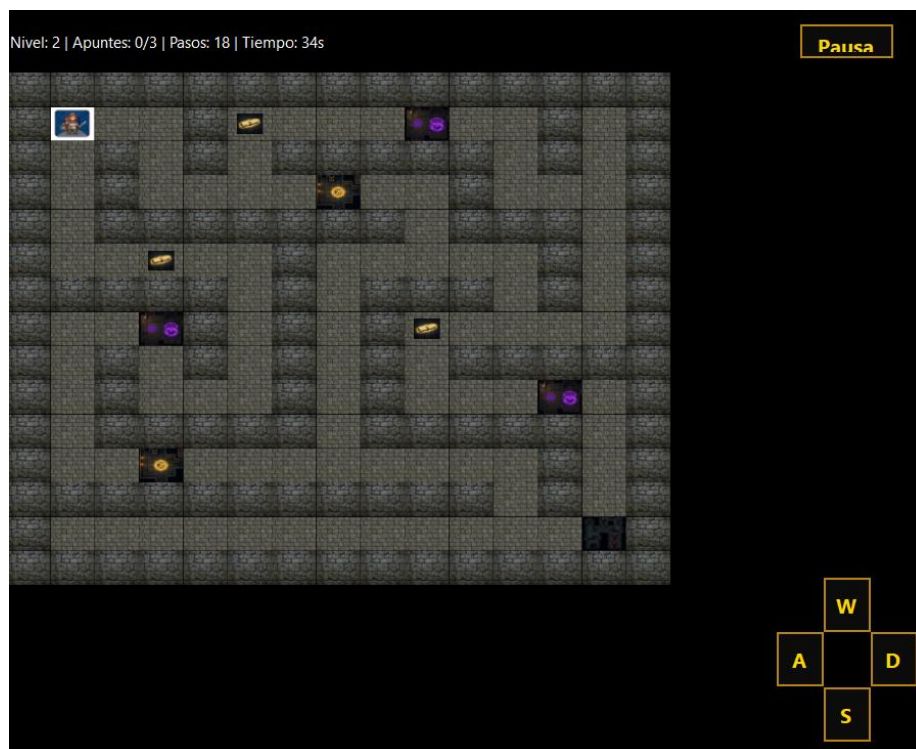


Figura 2 (Superior): Vista táctica del Nivel 2 (Desafío Intermedio). El laberinto se expande a una cuadrícula de 15x15 introduciendo bifurcaciones, trampas y múltiples elementos coleccionables (apuntes).

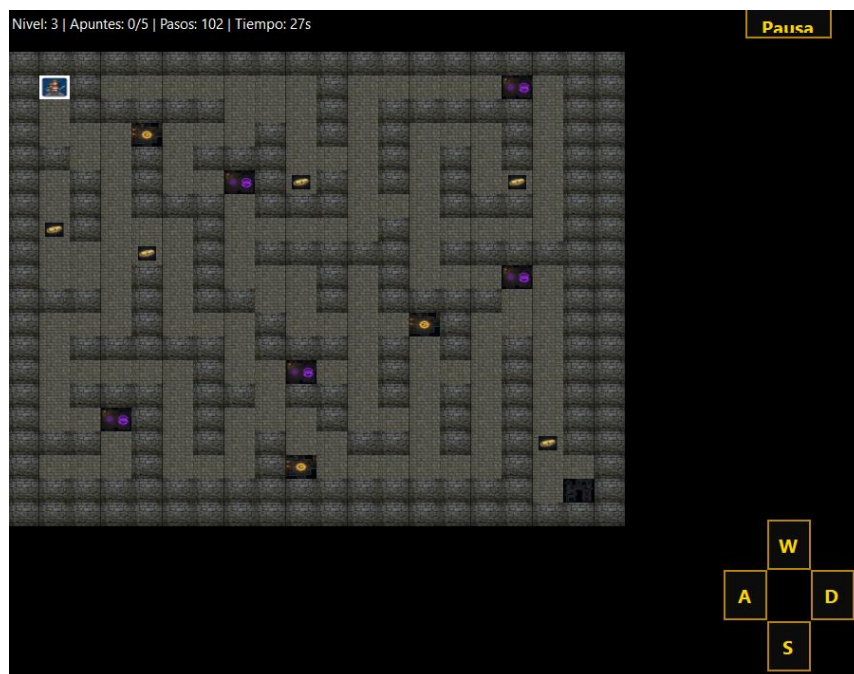


Figura 3: Diseño espacial del Nivel 3 (Laberinto Complejo). Entorno masivo de 20x20 repleto de callejones sin salida y rutas engañosas, diseñado para desorientar y desafiar la memoria espacial del usuario.

Evidencia de Código (Construcción del Nivel 1 y 2 en `Level.cs`):**C#**

```

switch (levelNumber)
{
    case 1:
        Grid = new int[10, 10]
        {
            { 1, 1, 1, 1, 1, 1, 1, 1, 1, 1 },
            { 1, 0, 0, 0, 1, 2, 0, 0, 0, 1 },
            { 1, 0, 1, 0, 1, 0, 1, 1, 0, 1 },
            { 1, 0, 1, 0, 0, 0, 0, 1, 0, 1 },
            { 1, 0, 1, 1, 1, 1, 0, 1, 0, 1 },
            { 1, 0, 0, 0, 0, 1, 0, 0, 0, 1 },
            { 1, 1, 1, 1, 0, 1, 1, 1, 0, 1 },
            { 1, 0, 0, 0, 0, 0, 0, 0, 0, 1 },
            { 1, 1, 1, 1, 1, 1, 1, 3, 1 },
            { 1, 1, 1, 1, 1, 1, 1, 1, 1 }
        };
        StartPosition = new Point(1, 1);
        ExitPosition = new Point(8, 8);
        break;

    case 2:
        Grid = new int[15, 15] // Matriz más amplia con múltiples desvíos
        {
            { 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1 },
            { 1, 0, 0, 0, 1, 2, 0, 0, 0, 4, 0, 0, 1, 0, 1 },
            { 1, 0, 1, 0, 1, 0, 1, 1, 1, 1, 1, 0, 1, 0, 1 },
            // ... (Se iteran 15 filas)
        };
        StartPosition = new Point(1, 1);
        ExitPosition = new Point(13, 13);
        break;
}

```

Personaje que se mueve con el teclado (y botones)

Explicación: Se implementó una interfaz gráfica mediante Windows Forms que detecta interacciones en tiempo real. Mediante la sobreescritura del método `ProcessCmdKey`, la aplicación captura las flechas del teclado (y W,A,S,D) e invoca la lógica de movimiento. Se incluyeron además botones visuales clickeables.

Evidencia de Código (Eventos de Teclado en `FormJuego.cs`):**C#**

```

// Captura de eventos del teclado en tiempo real
protected override bool ProcessCmdKey(ref Message msg, Keys keyData)
{
    if (_isPaused) return base.ProcessCmdKey(ref msg, keyData); // Validación de estado

    if (_gameManager.CurrentLevel != null)
    {
        switch (keyData)
        {
            case Keys.Up:
            case Keys.W: HandleMove(Direction.Up); return true;
            case Keys.Down:
            case Keys.S: HandleMove(Direction.Down); return true;
        }
    }
    return base.ProcessCmdKey(ref msg, keyData);
}

```

Mostrar Tiempo y Numero de Pasos (Estadísticas)

Explicación: Se actualiza una etiqueta en la interfaz visual (`Label`) combinando las propiedades numéricas del cronómetro y los contadores en cada turno o segundo transcurrido.

Evidencia de Código (`FormJuego.cs`):**C#**

```
private void gameTimer_Tick(object sender, EventArgs e)
{
    _gameManager.TimeElapsedSeconds++;
    UpdateStats();
}

private void UpdateStats()
{
    lblStats.Text = $"Nivel: {_gameManager.CurrentLevelNumber} | Apuntes:
{_gameManager.StarsCollected}/{_gameManager.CurrentLevel.TotalStars} | Pasos: {_gameManager.Steps} | Tiempo:
{_gameManager.TimeElapsedSeconds}s";
}
```

Puntos de Inicio y Salida

Explicación: Se requiere que cada laberinto tenga un punto donde nace el jugador y una meta. Se utilizaron variables tipo `Point` para guardar estas coordenadas lógicas. En cada movimiento, la POO permite a `GameManager` validar si el usuario llegó a la coordenada de salida.

Evidencia de Código (`Level.cs` y `GameManager.cs`):**C#**

```
// Constructor de Nivel (Level.cs)
StartPosition = new Point(1, 1);
ExitPosition = new Point(8, 8); // Coordenada de victoria dictada en la matriz

// Validación Lógica en cada paso de MovePlayer (GameManager.cs)
if (targetCell == 3) // La celda 3 representa la Salida en la matriz
{
    CheckWinCondition(); // Ejecuta lógica de victoria y avance
}
```

Recolección de Apuntes (Logica de Juego)

Explicación: Se obliga al jugador a explorar buscando elementos coleccionables (representados por el número 2 en la matriz).

Evidencia de Código (`GameManager.cs`):**C#**

```
if (targetCell == 2)
{
    StarsCollected++; // Registra la recolección
    CurrentLevel.Grid[newY, newX] = 0; // Remueve el ítem volviéndolo camino vacío
}
```

Interacciones con Trampas y Potenciadores

Explicación: El jugador pisa celdas trampa (4) que enojan al enemigo acelerándolo, o toma potenciadores (5) que congelan temporalmente al enemigo.

Evidencia de Código (`GameManager.cs`):**C#**

```
else if (targetCell == 4) // TRAMPA (Maldición)
{
    CurrentLevel.Grid[newY, newX] = 0;
    _enemyTimer.Interval = Math.Max(200, _enemyTimer.Interval - 100); // Acelera al enemigo
}
else if (targetCell == 5) // POTENCIADOR (Boost)
{
    CurrentLevel.Grid[newY, newX] = 0;
    _enemyFrozenTime = 5; // Congela al enemigo 5 turnos
}
```

Enemigo Inteligente (Monstruo Acosador)

Explicación: Un enemigo en constante movimiento utiliza algoritmos (como BFS) dentro de un `Timer` independiente para encontrar el camino al jugador esquivando las paredes de la matriz.

Evidencia de Código (`GameManager.cs`):

```
C#
private void MoveEnemy()
{
    // ... Cálculo de posición buscando al jugador ...
    // Choque mortal:
    if (EnemyPosition == new Point(Player1.X, Player1.Y))
    {
        OnLevelCompleted?.Invoke(false); // Dispara Game Over
    }
}
```

Niebla de Guerra y Nivel Oculto Hardcore

Explicación: Hay una variable modo Hardcore. Si se activa, las zonas alejadas del jugador se pintan de negro calculando la distancia euclidiana. Adicionalmente desbloquea un Nivel 4 hiper complejo de 25x25.

Evidencia de Código (`FormJuego.cs` y `Level.cs`):

```
C#
// NIEBLA DE GUERRA EN FORM JUEGO
if (_isHardcore)
{
    // Teorema de pitágoras: Distancia geométrica al jugador
    double distance = Math.Sqrt(Math.Pow(x - _gameManager.Player1.X, 2) + Math.Pow(y - _gameManager.Player1.Y, 2));
    if (distance > FogRadius)
    {
        e.Graphics.FillRectangle(Brushes.Black, rect); // Ciega al usuario
    }
}

// NIVEL 4 OCULTO EN LEVEL.CS
case 4:
    Grid = new int[25, 25] { /* ... */ }; // Matriz inmensa
    TotalStars = 8;
    break;
```

Subida de Avatar Personalizado (Pantalla de Menu)

Explicación: En la pantalla `FormMenu`, el jugador no está limitado a un ícono fijo. Puede importar imágenes en formato JPG/PNG desde su PC y jugar con ellas.

Evidencia de Código (`FormMenu.cs`):

```
C#
private void btnUploadAvatar_Click(object sender, EventArgs e)
{
    using (OpenFileDialog ofd = new OpenFileDialog())
    {
        ofd.Filter = "Image Files|*.jpg;*.jpeg;*.png;*.bmp";
        if (ofd.ShowDialog() == DialogResult.OK)
        {
            _avatarImage = Image.FromFile(ofd.FileName); // Carga la imagen local
            pbAvatar.Image = _avatarImage;
        }
    }
}
```

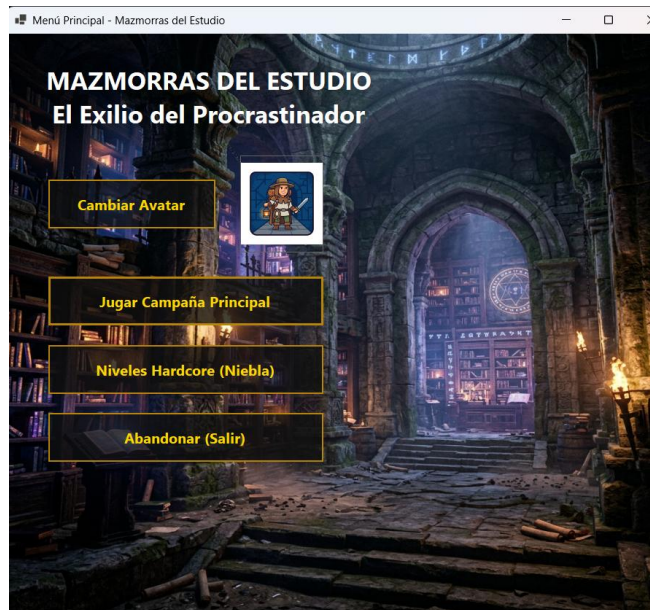



Figura 4: Pantalla del Menú Principal (FormMenu). Muestra la interfaz gráfica de inicio con el título del juego "Mazmorras del Estudio", el avatar predeterminado habilitado y las distintas opciones de juego.

Texturas y Efecto Visual Pulsante (Pantalla de Juego)

Explicación: En lugar de colores sólidos, dibujamos imágenes que se escalan dinámicamente. La meta incluye una lógica matemática senoidal para contraerse y expandirse (Glow effect).

Evidencia de Código (FormJuego.cs):

```
C#
// Escalado de la salida (Meta) con un efecto tipo 'Glow'
int targetSize = (int)(CellSize * glowScale);
int offset = (CellSize - targetSize) / 2;
Rectangle targetRect = new Rectangle(x * CellSize + offset, y * CellSize + offset, targetSize, targetSize);
e.Graphics.DrawImage(Properties.Resources.salida, targetRect); // Dibuja la textura animada
```

Sistema de Audio y Banda Sonora

Explicación: Implementación nativa de `System.Media.SoundPlayer` para música de fondo en los niveles y efectos de terror en situaciones críticas.

Evidencia de Código (AudioPlayer.cs):

```
C#
public void PlayMusic()
{
    _musicPlayer.Stream = Properties.Resources.scary_music;
    _musicPlayer.PlayLooping();
}
```

Pantallas de Historia y Finales Múltiples (FormHistoria)

Explicación: Se añadió una pantalla cinemática al final. Revisa el estado de la partida terminada (Si el jugador logró reunir el 100% de apuntes) y carga un final "Perfecto" o un final "Normal" alentando la rejugabilidad.

Evidencia de Código (FormHistoria.cs):

C#

```
public FormHistoria(bool perfectEnding)
{
    InitializeComponent();

    if (perfectEnding)
        txtStory.Text = "¡Felicidades! Lograste encontrar todos los apuntes perdidos...";
    else
        txtStory.Text = "Lograste escapar con vida, pero te faltaron apuntes de clase por recuperar...";
}
```

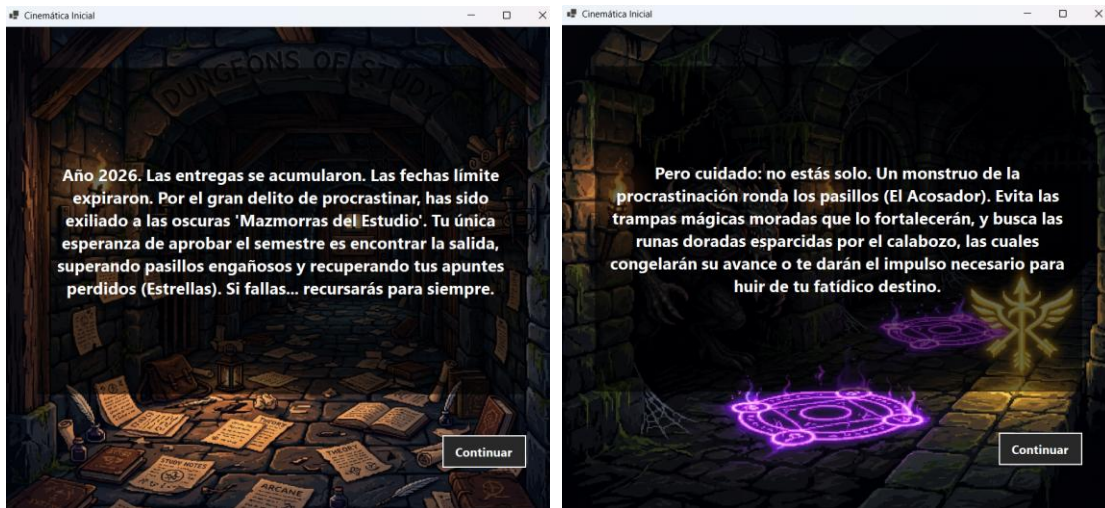


Figura 5: Cinemática de Introducción. Pantalla narrativa (FormHistoria) que establece el contexto del juego (Año 2026), el objetivo principal y una advertencia sobre los enemigos que persiguen al jugador.

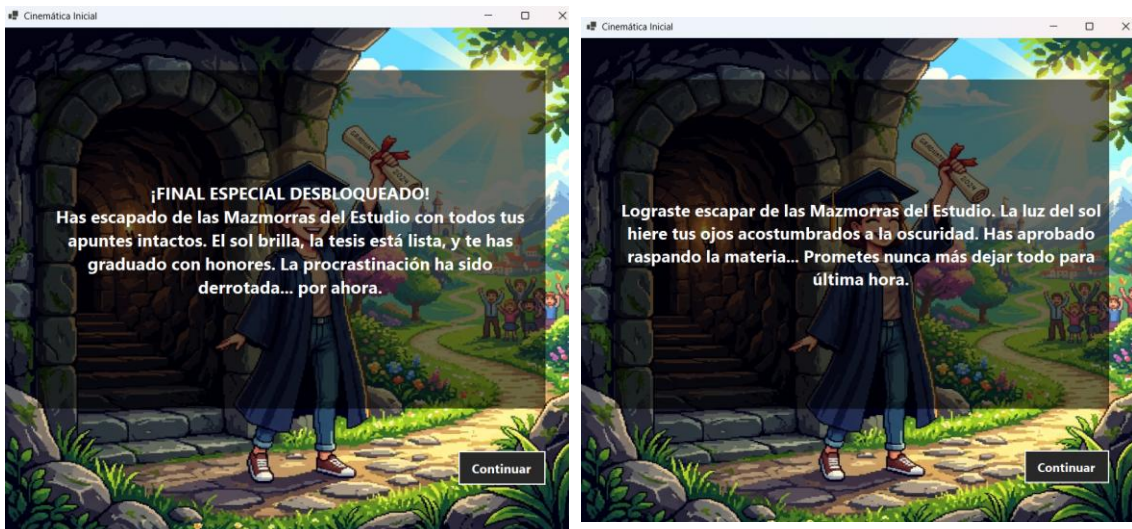


Figura 6> Pantalla de Final Perfecto. Desenlace especial desbloqueado dinámicamente si el sistema detecta que el jugador logró escapar recolectando el 100% de los apuntes perdidos.

Figura 7> Pantalla de Final Normal. Desenlace estándar que se muestra cuando el jugador alcanza la salida, pero falló en la recolección total de los apuntes, reflejando que aprobó

Conclusion

El desarrollo del proyecto "Laberinto Interactivo con Niveles" ha representado un ejercicio integral y sumamente enriquecedor que nos ha permitido llevar a la práctica los conceptos teóricos fundamentales de la programación. A lo largo del diseño y la codificación de este aplicativo, logramos demostrar de manera fehaciente la importancia de comprender y aplicar correctamente las estructuras de datos, haciendo especial énfasis en el uso de matrices bidimensionales. Estas matrices no solo sirvieron como un método eficiente de almacenamiento de datos, sino que se convirtieron en el eje central que modeló toda la estructura espacial, geométrica y lógica del entorno virtual, demostrando la versatilidad de los arreglos para simular planos cartesianos.

Se cumplieron con éxito todos los objetivos trazados en la rúbrica de evaluación. Logramos construir una interfaz gráfica intuitiva y funcional mediante la tecnología de Windows Forms, superando el desafío de la renderización en tiempo real. Esto implicó un manejo avanzado de eventos (Events), donde el control del teclado y del ratón, junto con el uso preciso de temporizadores (Timers), posibilitó una interacción fluida y constante entre el usuario y la aplicación. Además, la estructuración del código se rigió bajo los pilares de la Programación Orientada a Objetos (POO), lo que garantizó la separación de responsabilidades (MVC parcial) entre las clases visuales y los controladores lógicos (GameManager, Level, etc.), resultando en un sistema sumamente limpio, escalable y libre de código redundante.

Más allá de los requisitos básicos de un mapa estático, el equipo asumió el reto de implementar mecánicas de mayor complejidad algorítmica. La incorporación de un enemigo regido por inteligencia artificial para la persecución del jugador, sumada al diseño de elementos interactivos dinámicos (como trampas, potenciadores y objetos coleccionables) y efectos visuales/sonoros inmersivos, eleva la aplicación de un simple programa académico a un prototipo de videojuego completo y robusto. La capacidad de guardar estados lógicos para desembocar en distintos finales de historia o modos de juego extra, pone en evidencia el profundo control adquirido sobre el flujo de ejecución del programa.

En definitiva, la elaboración de este proyecto ha consolidado fuertemente nuestras habilidades en el lenguaje C#. Nos ha brindado una visión madura y estructurada sobre cómo la matemática computacional, la validación estricta de datos y el diseño interactivo se unen para crear software de calidad. El aprendizaje obtenido trasciende el manejo de sintaxis, dotándonos de valiosas competencias en la resolución lógica de problemas, la optimización de algoritmos y el trabajo metódico; cimientos indispensables para nuestros futuros retos profesionales en el ámbito del desarrollo de software.

Referencias Bibliograficas

1. Microsoft Corporation. (2023). *Documentación de C# y .NET*. Recuperado de <https://learn.microsoft.com/es-es/dotnet/csharp/>
2. Microsoft Corporation. (2023). *Windows Forms Documentation*. Recuperado de <https://learn.microsoft.com/es-es/dotnet/desktop/winforms/>
3. Albahari, J., & Albahari, B. (2022). *C# 10 in a Nutshell: The Definitive Reference*. O'Reilly Media.
4. Freeman, A. (2020). *Pro C# 9 with .NET 5: Foundational Principles and Practices in Programming*. Apress.